Q. 1 — Understanding a Data Pipeline

- Source: Customer and order files
- Ingestion: FTP/ Multi file drop
- Raw: Data in the files with no change
- Staging: Data read, cleaned and validated in Python
- Transformation: Calculations based on business logics
- Analytics: aggregated tables storefd and fed directly to dashboards/reports

Q. 2 — Compare ETL and ELT

Area	ETL	ELT
When transformation happens	Before the data is stored	When the data is read
Where transformation usually runs	Python scripts along with ingestion logic	SQL queries when the data is read
Suitable example	API request → Enforce schema and drop unnecessary keys → Store in DB	Ingestion → Save raw data (bronze) → Generate OLAP table (silver)

Q. 3 — Practical: Rewrite an ETL Flow as ELT

Extract JSON files (E)

↓

Load JSON files into the lakehouse (L)

↓

Query, clean and standardise with SQL (T)

Q. 4 — Scenario: Choose ETL or ELT

- ETL is preferred
- Reason 1: The scenario states that the target database must contain only only valid records; hence, inconsistent dates and prices must be handled first before savings

- Reason 2: The files come from different suppliers which would likely have different structures, so it is useful to validate (transform) before loading.
- Trade-off: Information loss might occur if the transformation logic is not strict enough

Q. 5 — MCQ: Pipeline Purpose

What is the main purpose of a data pipeline?
B. Move and process data through defined stages

Q. 6 — MCQ: Raw Layer

Which statement best describes a raw layer?
B. It stores data close to its original form

Q. 7 — MCQ: ETL Order

Which sequence represents ETL?
A. Extract → Transform → Load

Q. 8 — MCQ: ELT Transformation

Where does transformation usually happen in ELT?
B. After loading into the target platform

Q. 9 — MCQ: Best Fit

Which scenario most strongly supports ELT?
A. A modern warehouse can store raw data and run large SQL transformations

Q. 10 — Batch and Streaming Processing

Use Case	Choice
Daily sales report	Batch
Payment-fraud alert	Streaming
Hourly stock snapshot	Batch
Immediate delivery notification	Streaming

Q. 11 — Practical: Simulate Batch and Streaming Events

```
import time
order_events = [
    {"order_id": '001', "customer_id": 10, "product_id": 1, "quantity": 2,
"price": 100 },
    {"order_id": '002', "customer_id": 50, "product_id": 2, "quantity": 1,
"price": 10 },
    {"order_id": '003', "customer_id": 10, "product_id": 2, "quantity": 2,
"price": 20 },
    {"order_id": '004', "customer_id": 22, "product_id": 4, "quantity": 5,
"price": 50 },
    {"order_id": '005', "customer_id": 15, "product_id": 3, "quantity": 3,
"price": 30 },
]

print("---- BATCH ----")
print(order_events)
print("---- STREAM ----")
for event in order_events:
    print(event)
    time.sleep(1)
```

Q. 12 — Practical: Ingest a Customer File into Raw Storage

```
import pandas as pd

raw_path = "data/raw/customers.csv"
source_path = "data/source/customers.csv"

df = pd.read_csv(raw_path)
(df.copy()).to_csv(source_path, index=False)

raw_df = pd.read_csv(source_path)
print(f"Ingestion of {len(raw_df)} rows complete")
```

Q. 13 — Practical: Clean Raw Customers into Staging

```
import pandas as pd
raw_df = df.copy()
raw_df["name"] = raw_df["name"].str.strip()
raw_df["email"] = raw_df["email"].str.lower()
raw_df.to_csv("data/staging/customers.csv", index= False)
print(raw_df)
```

Q. 14 — MCQ: Daily Report

Which processing style best fits a report produced once every night?
A. Batch

Q. 15 — MCQ: Immediate Alert

Which processing style best fits an immediate fraud alert?
B. Streaming

Q. 16 — MCQ: Ingestion

What does an ingestion job normally do?
A. Moves data into a pipeline

Q. 17 — MCQ: Staging

Which task belongs most naturally in staging?
A. Trim names and validate required fields

Q. 18 — MCQ: Layer Benefit

Why keep raw and staging data separate?
A. To preserve the original input while creating a cleaned version

Q. 19 — Pipeline Dependencies

- Ingest_customers → clean_customers → build_customer_summary → publish_report
- If the “clean_customers” task fails, the pipeline fails immediately as the “build_customer_summary” task would have errors due to handling raw data
- Each pipeline task validates the next task and needs to be completed for the next task to run

Q. 20 — Practical: Stop a Pipeline after Failure

```
def ingest():
    print("ingest: success")

def clean():
    try:
        df = pd.read_csv("data/raw/customers.csv")
        print("clean: success")
    except FileNotFoundError as e:
        print(e)

def publish():
    print("publish: success")


steps = {
    "ingest" : ingest,
    "clean" : clean,
    "publish" : publish
    }

for step, fnx in steps.items():
    try:
        fnx()
    except Exception as error:
        print(error)
        break
else:
    print("Pipeline finished successfully")
```